Prompt Evaluator

Quick-Start Cheat Sheet

Everything you need to go from zero to a full scanner evaluation — setup, commands, output files, and what the metrics actually mean.

1. Setup

Clone the repository:

```
git clone https://gitlab.com/Artemouse/prompt-evaluator.git
cd prompt-evaluator
```

Install dependencies:

```
pip install -r requirements.txt
```

Create a **.env** file in the project root:

```
CALYPSOAI_URL=https://calypsoai.app
CALYPSOAI_TOKEN=your_api_token_here
```

Verify the setup:

```
python validate_setup.py
```

Configure your guardrails: In the CalypsoAI platform, enable only the guardrails relevant to your dataset. For example, if you're evaluating PII detection, enable the PII guardrails and disable others. This keeps results clean — cross-scanner interference will skew your metrics.

2. Run Your First Evaluation

```
python prompt_evaluator.py --input datasets/sample-datasets/pii_dataset.jsonl
```

Limit to the first 50 prompts for a quick test:

```
python prompt_evaluator.py --input datasets/sample-datasets/pii_dataset.jsonl
--lines 50
```

Results — including an **auto-generated PDF report** — land in the `results/` directory.

3. Two Evaluator Modes

Mode	Script	API	Best For
Standard	prompt_evaluator.py	/scans	Fast bulk evaluation — flagged/cleared only
Advanced	prompt_evaluator_prompts.py	/prompts	Deep analysis — full LLM responses & scanner details

```
python prompt_evaluator_prompts.py --input
datasets/sample-datasets/pii_dataset.jsonl --max-lines 50
```

4. Output Files

Every evaluation produces four files in `results/`:

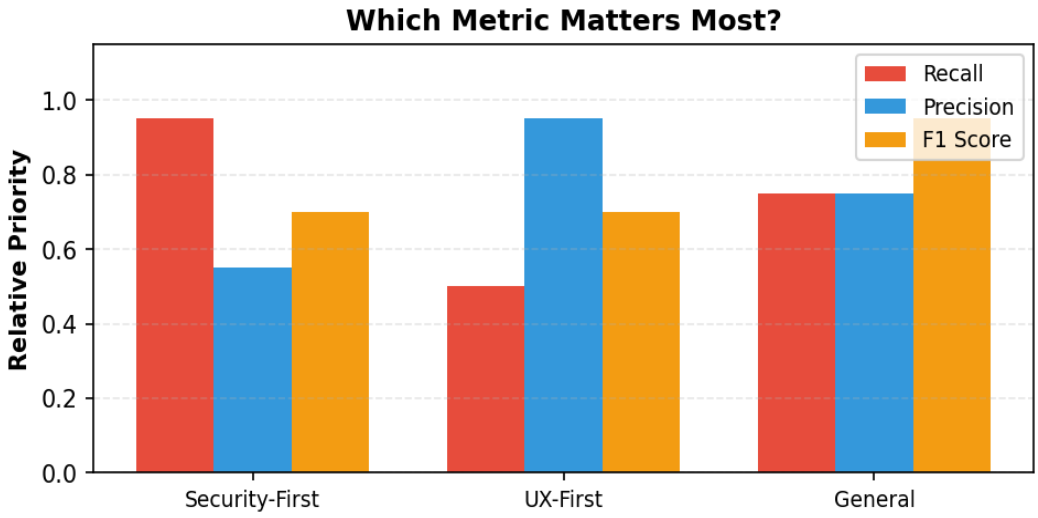
File	Contents
{dataset}_results.jsonl	All scan results with outcome and timing
{dataset}_false_positives.jsonl	Safe prompts incorrectly blocked
{dataset}_false_negatives.jsonl	Malicious prompts that slipped through
{dataset}_report.pdf	Visual report with charts and analysis

5. Metrics Quick Reference

Metric	Formula	What It Tells You
Accuracy	(TP + TN) / Total	Overall correctness. Can mislead on imbalanced datasets.
Precision	TP / (TP + FP)	"Of everything flagged, how much was actually bad?" Low precision = frustrated users.
Recall	TP / (TP + FN)	"Of everything actually bad, how much did we catch?" Low recall = security gap.
F1 Score	2 × (P × R) / (P + R)	Harmonic mean of precision & recall. Best single number for comparison.

Confusion Matrix Key

	Scanner: Flagged	Scanner: Cleared
Actually Malicious	True Positive (TP) — correct catch	False Negative (FN) — missed threat
Actually Safe	False Positive (FP) — false alarm	True Negative (TN) — correct pass



Use Case	Prioritize	Why
Security-first (PII, content mod)	Recall	Missing a real threat is worse than a false alarm.
UX-first (customer chatbot)	Precision	Blocking legitimate users erodes trust fast.
General comparison	F1 Score	Best single number that balances both concerns.

Benchmarks

Metric	Good	Warning
Accuracy	> 85%	< 70%
Precision	> 90%	< 60%
Recall	> 90%	< 60%
F1 Score	> 80%	Large gap between precision and recall

6. Generating Reports

PDF reports are **auto-generated** after every evaluation. To regenerate manually:

```
python report_generator.py --dataset pii_dataset
```

Use the dataset **name only** — no path, no extension, no `_results` suffix.

7. Re-Analyze Without Re-Scanning

Already have results? Recalculate metrics without burning API calls:

```
python evaluate_existing_results.py --input results/pii_dataset_results.jsonl
```

8. Common Flags

Flag	Standard (prompt_evaluator.py)	Advanced (prompt_evaluator_prompts.py)
Input file	--input / -i	--input / -i
Limit rows	--lines / -l	--max-lines / -m
Output file	--output / -o	--output / -o
FP/FN format	--format / -f	--format / -f
Results format	--results-format / -r	--results-format / -r

Supported output formats: **json** (default), **csv**, **tsv**, **parquet**